

SA-MIRI Course Schedule (19/11 Tentative)

Practical Session	acronim	Book Chapter included
Pi	SCALING PT	task 12.1 – Reproducing distributed training results on mn5 task 12.2 – Analyze and compare scaling efficiency task 12.3 – Investigate diminishing returns in training time task 12.4 – Find the sweet spot for your use case
Pk	OPT & SCAL LLM	task 15.1 – Baseline experiment using facebook/opt-1.3b task 15.2 – Finding the out-of-memory limit task 15.3 – Mixed precision training task 15.4 – Model precision task 15.5 – Increasing batch size with model precision task 15.6 – Enabling flash attention task 15.7 – Increasing batch size with flash attention task 15.8 – Using the liger kernel task 15.9 – Augmenting batch size due to liger kernels task 15.10 – Scaling on multiple gpus task 15.11 – Final reflections
Pl	FUTURE	task 16.1 – Mapping the pendulum shifts of the triad task 16.2 – Exploring the new giants of compute task 16.3 – Powering the future of ai task 16.4 – Charting the role of quantum in future supercomputing task 16.5 – From data farms to ai preserves task 16.6 – Embodiment as a source of learning

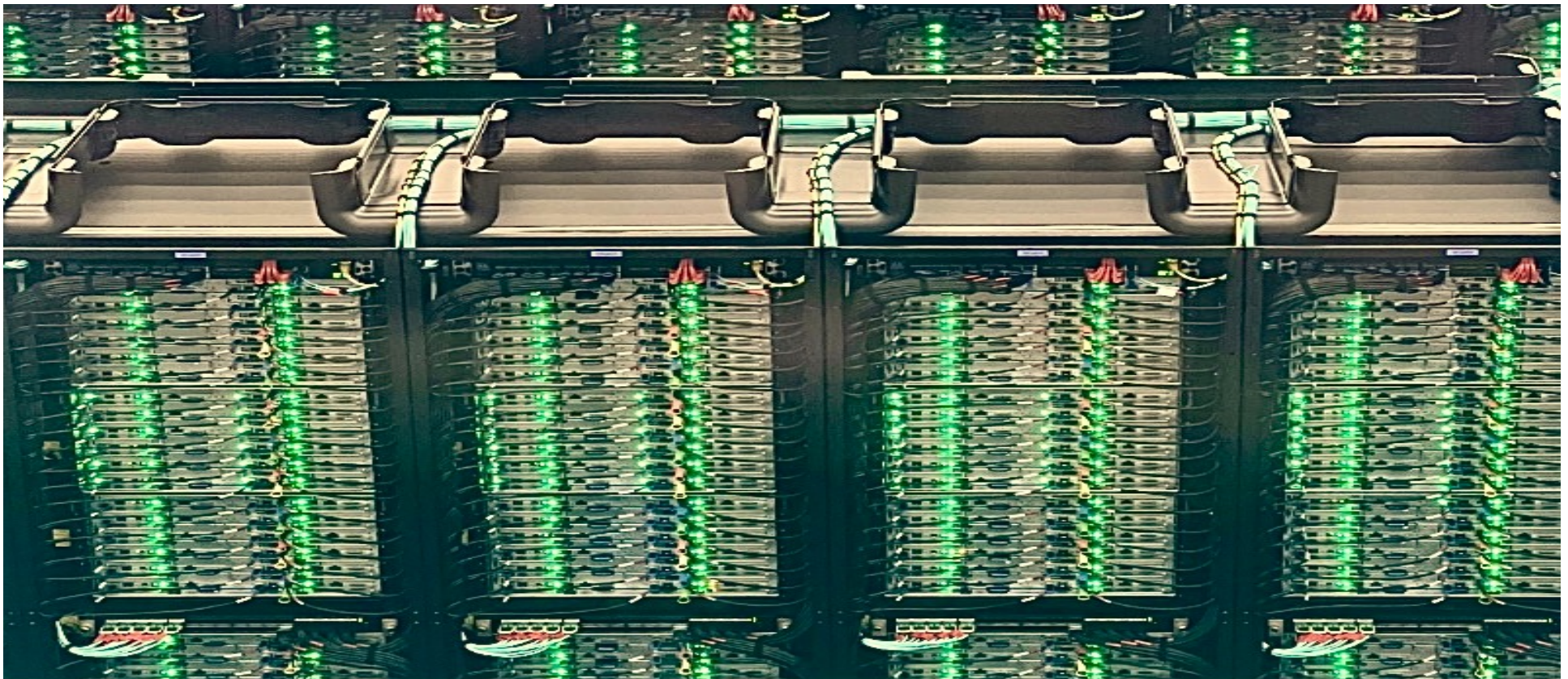
SA-MIRI Course Schedule (19/11 Tentative)

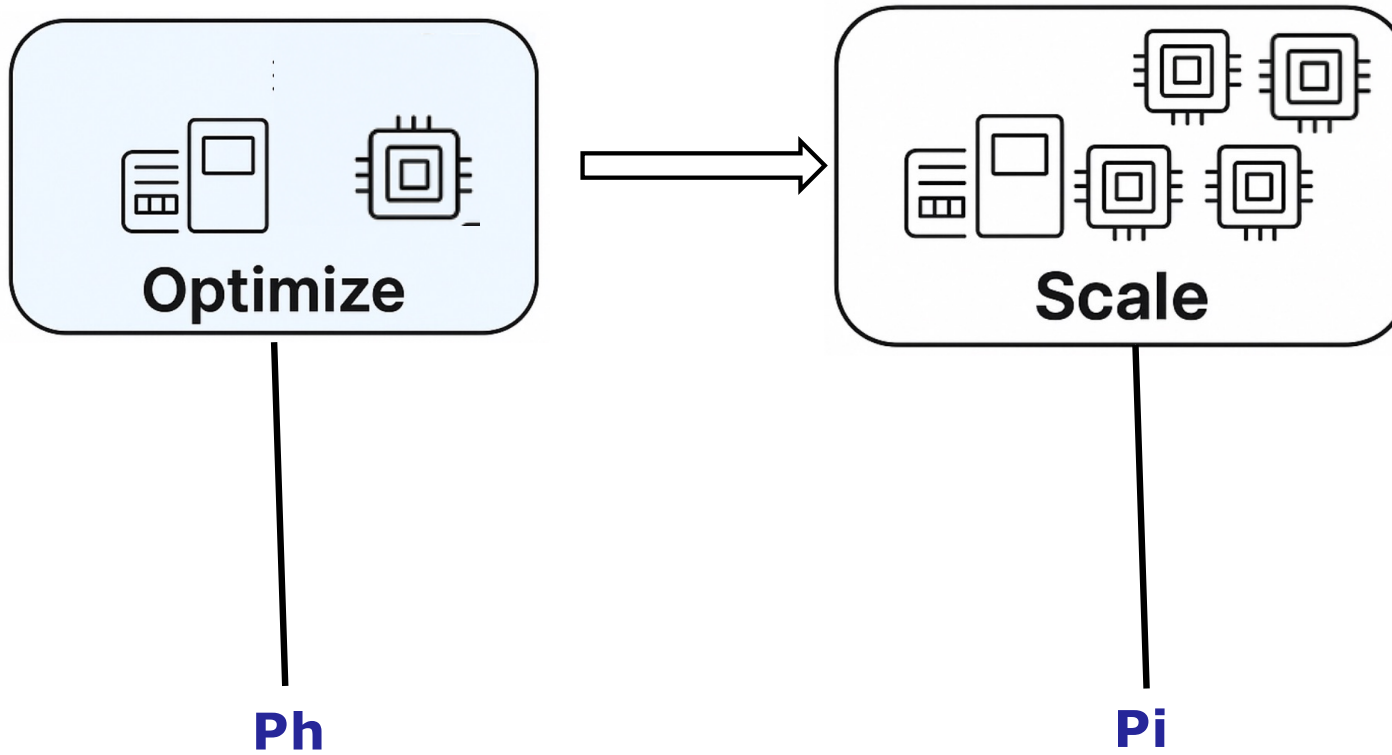
day	Wednesday	day	Friday
19/11	12. Parallelizing Model Training with Distributed Data Parallel (Pi) Ph/Pi Presentation Pg	21/11	Pi
26/11	Honoris Causa Dr. Oriol Vinyals (no class)	28/11	Pi
03/12	13. Introduction to Large Language Models 14. End-to-End Large Language Models Workflow Presentation Ph	05/12	Pk
10/12	15. Exploring Optimization and Scaling of LLMs Presentation Pi	12/12	Pk*
17/12	Presentation P1 (16. Looking Forward: HPC and AI Futures) All groups will present	19/12	Pk* “attendance not required (for students returning home)”

12. Parallelizing Model Training with Distributed Data Parallel

Supercomputing for Artificial Intelligence
Foundations, Architectures, and Scaling Deep Learning Workloads

Jordi **TORRES.AI**



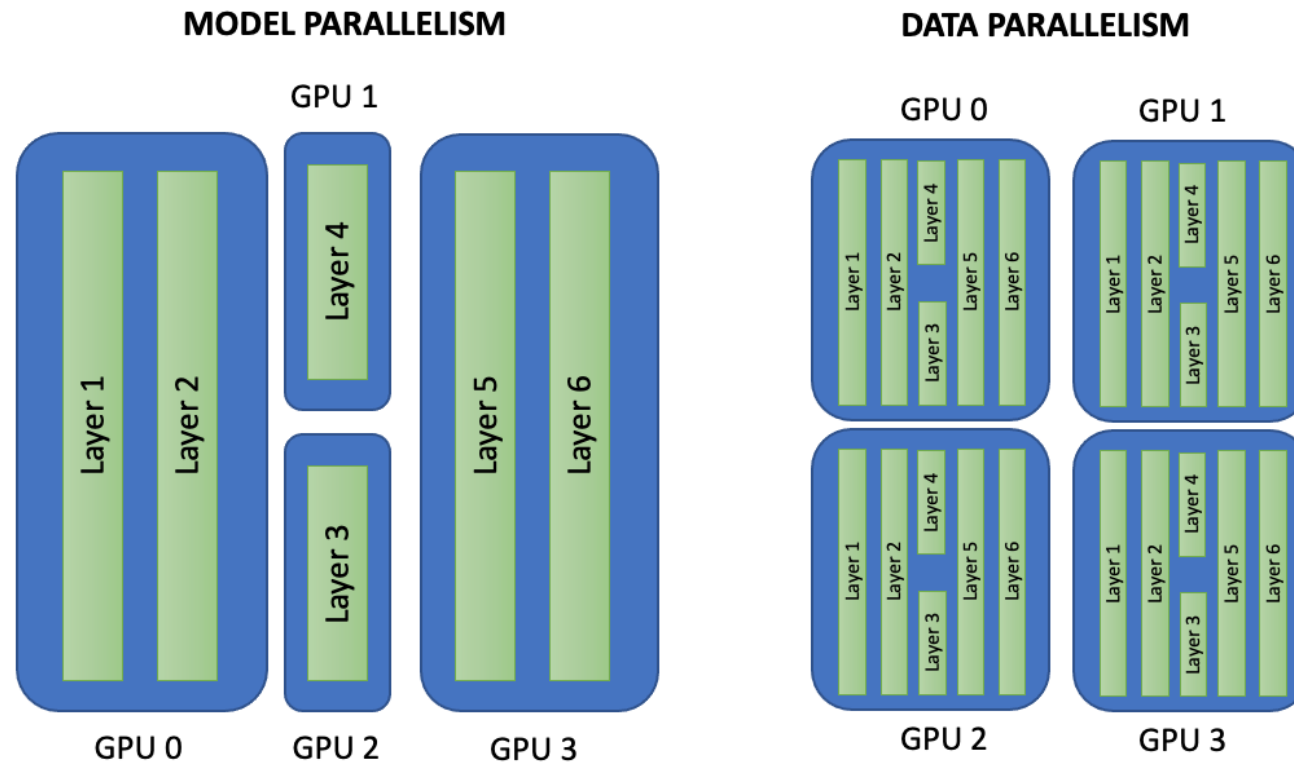




How Computation is Distributed

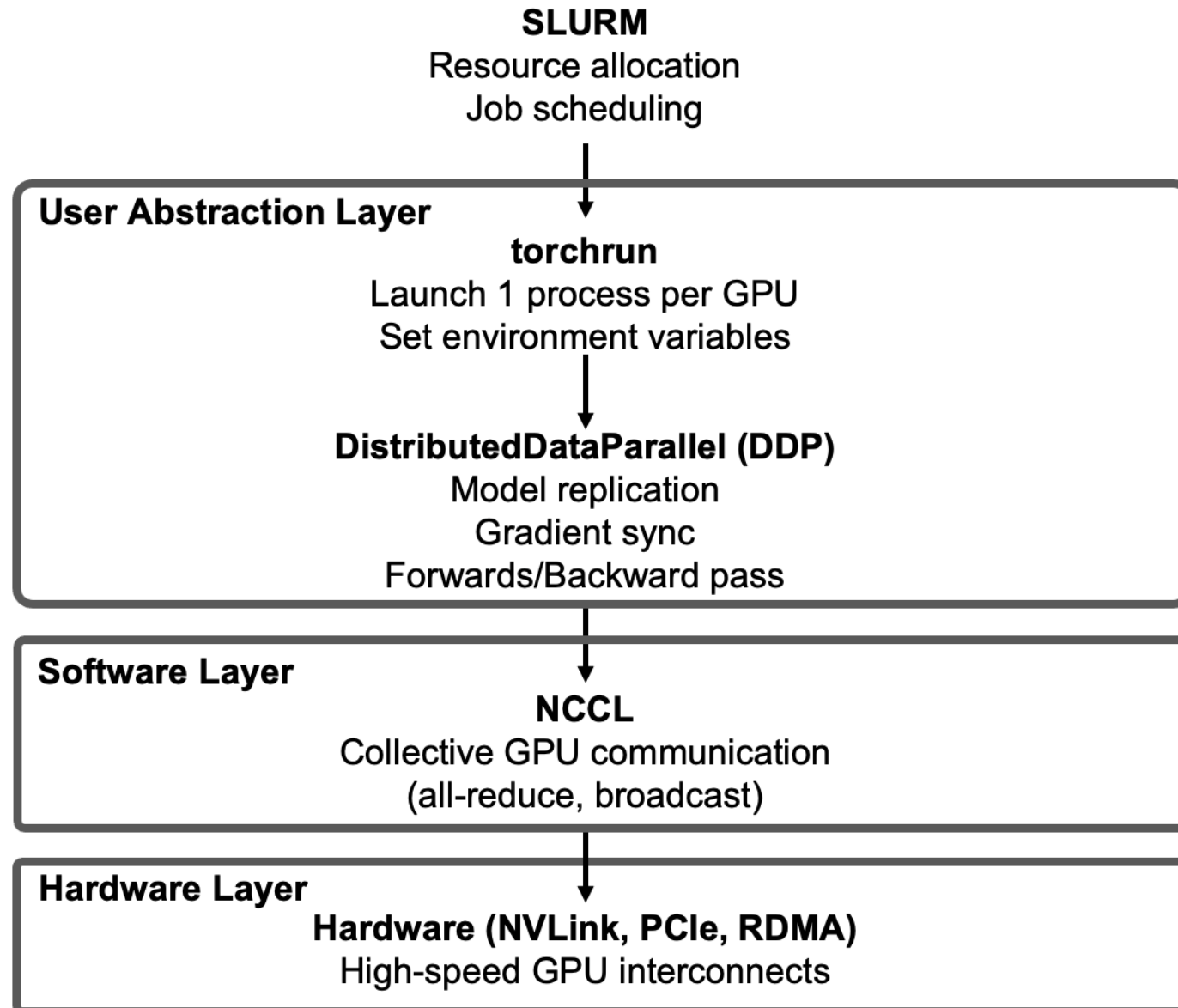
From Single-GPU to Distributed Training

- We use data parallelism, the simplest and most widely used scaling approach.



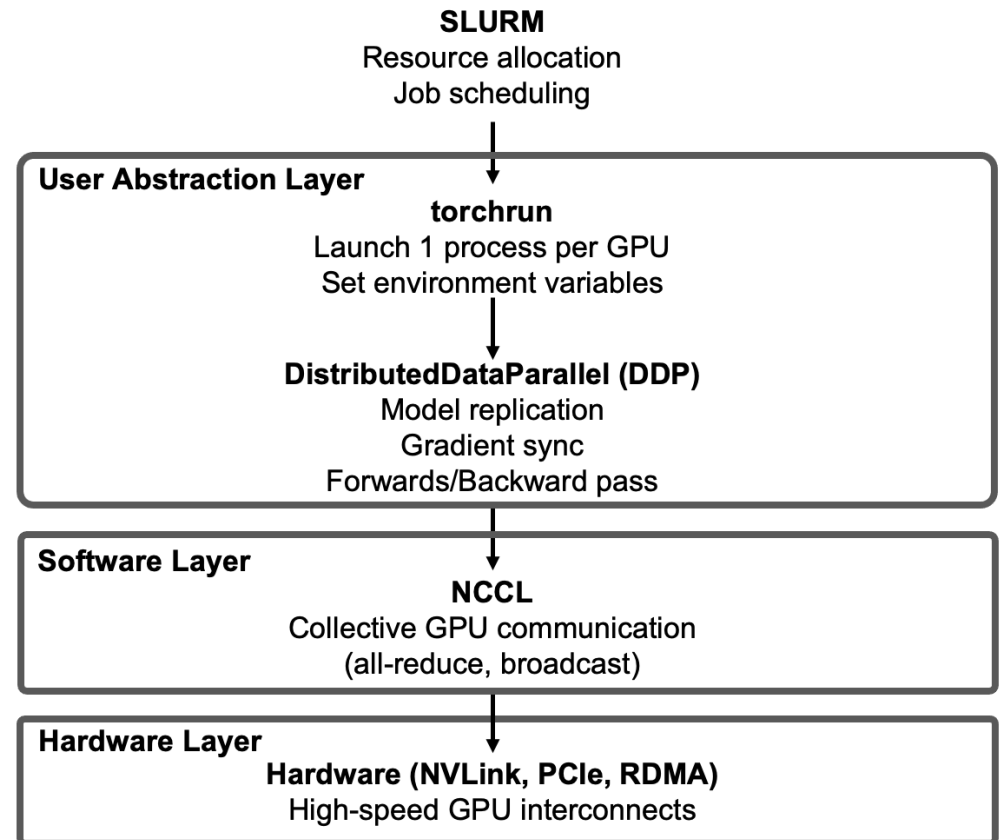
- Goal: understand *how* training accelerates — not improve accuracy.

The Distributed Training Software Stack



Execution Flow During a Distributed Job

- **SLURM / srun**
Allocates GPUs + nodes, launches distributed execution.
- **torchrun**
Spawns one process per GPU, sets ranks and environment variables.
- **DistributedDataParallel (DDP)**
Replicates model, partitions data, runs forward/backward, synchronizes gradients.
- **NCCL**
Performs all GPU-to-GPU communication (all_reduce, broadcast...).



Introducing torchrun

■ torchrun: The Entry Point for DDP Training

- Launches one Python process per GPU.
- Sets all distributed environment variables automatically.
- Manages the rendezvous (rdzv) phase so processes can find each other.
- Works hand-in-hand with DDP to initialize the training session.

```
torchrun \  
  --nproc_per_node=4 \  
  --nnodes=2 \  
  --rdzv_backend=c10d \  
  --rdzv_endpoint=node0:29400 \  
train.py
```

What is Distributed Data Parallel?

■ DDP: The Workhorse of Scaling

- Each GPU gets:
 - ✓ its own Python process
 - ✓ a full model replica
 - ✓ a different input batch
- All GPUs run forward + backward independently.
- Gradients are synchronized using NCCL all_reduce.
- Ensures consistent model updates across all processes.

The DDP Workflow

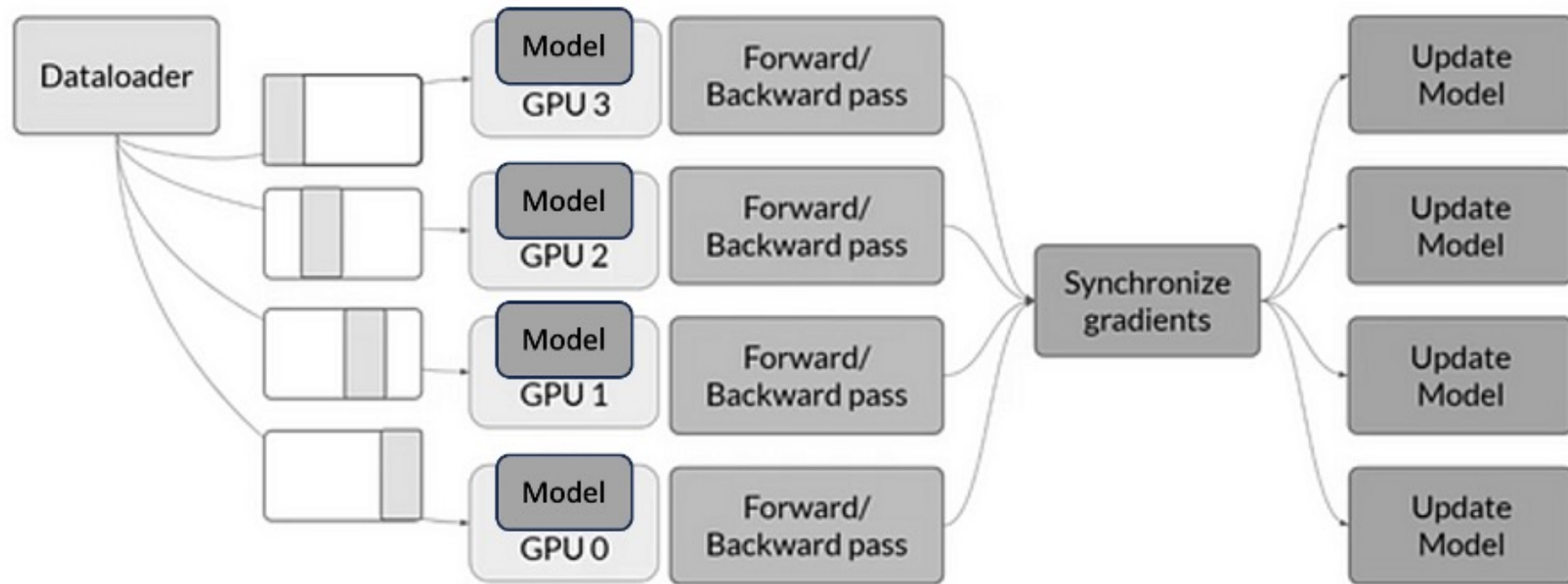


Figure 12.2 – Data-parallel training with DDP on a single node with four GPUs. Each GPU runs a separate process and receives different input batches. After the forward and backward passes, gradients are synchronized across processes to ensure consistent model updates.

- Each GPU works on a separate batch.
- After backward pass, gradients are aggregated across processes.
- Each model replica is updated with synchronized parameters.

DDP Across Multiple Nodes

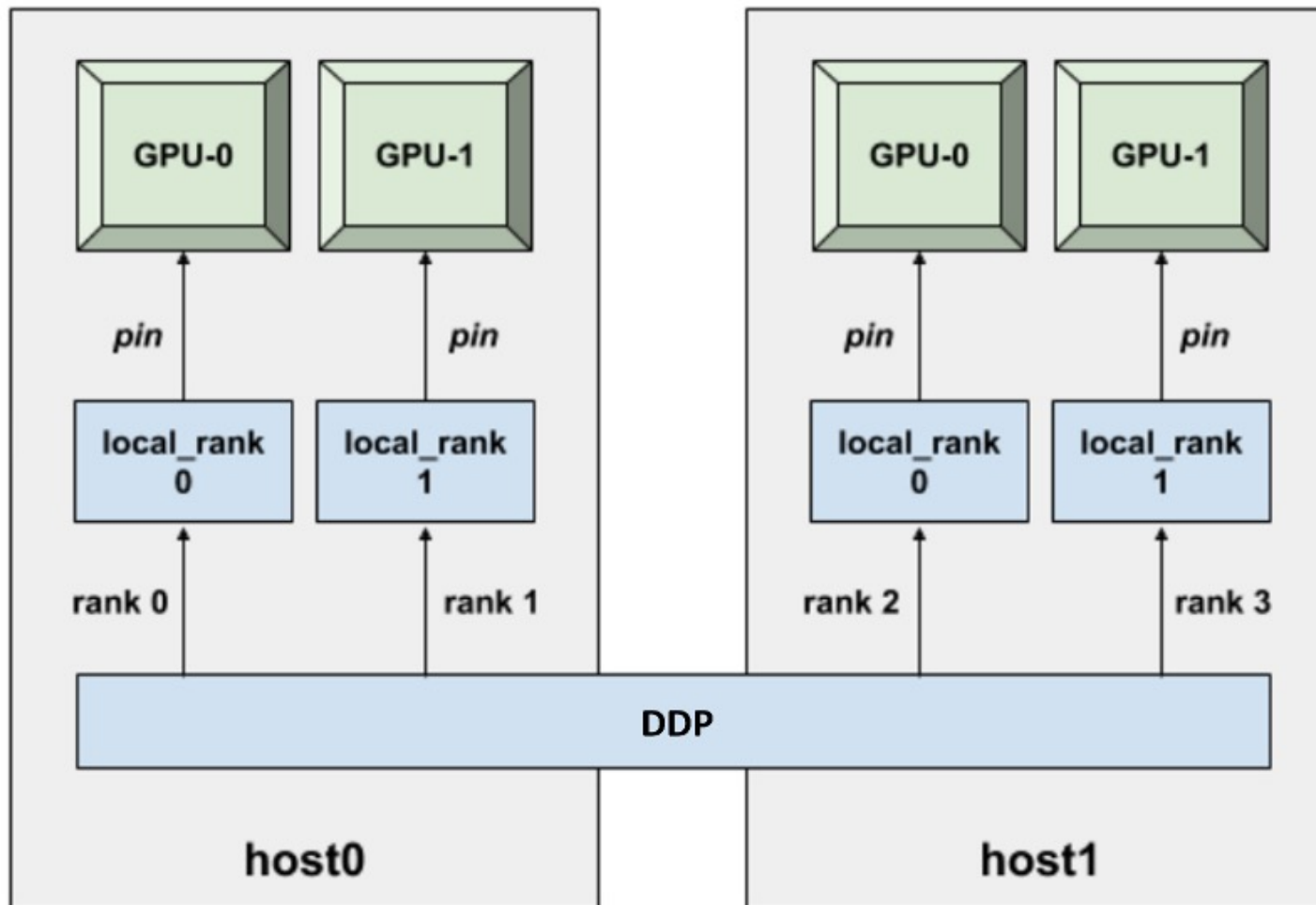
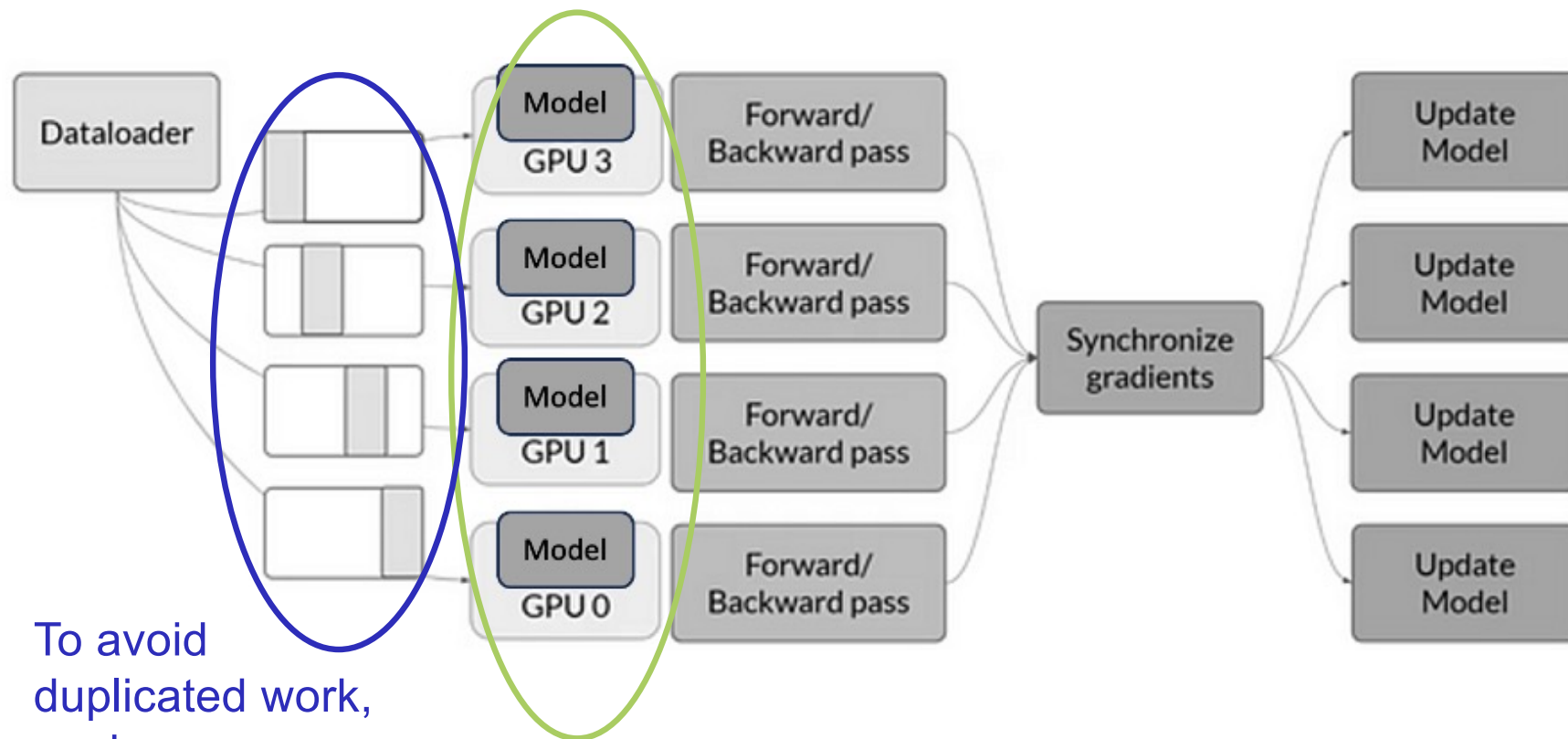


Figure 12.3 – DDP setup across two nodes, each with two GPUs. Each GPU is assigned a unique process. **local_rank** identifies the GPU index within a node, while **rank** corresponds to the global identifier across all processes. Communication and synchronization are handled transparently by DDP. (Image source: Horovod).



Dataset Sharding and Preprocessing in DDP

Dataset Sharding in DDP



To avoid duplicated work, each process must receive different data.

In DDP, each GPU runs its own process and holds its own replica of the model.

DistributedSampler: The Key Tool

■ torch.utils.data.DistributedSampler

- PyTorch does not automatically split datasets in DDP.
- DistributedSampler handles:
 - dataset partitioning across processes
 - shuffling before each epoch
 - assigning each process a unique subset

```
sampler = torch.utils.data.distributed.DistributedSampler(  
    train_dataset,  
    num_replicas=world_size,  
    rank=global_rank,  
    shuffle=True  
)  
  
train_loader = torch.utils.data.DataLoader(  
    train_dataset,  
    sampler=sampler,  
    batch_size=batch_size,  
    num_workers=4  
)
```

Clean Logging and Checkpointing

■ Avoiding Output Flooding in DDP

- DDP spawns one process per GPU.
- If all processes print → console flooding.
- **Standard convention: only `global_rank == 0` prints or saves checkpoints.**

■ Code:

```
if global_rank == 0:  
    print(...)
```

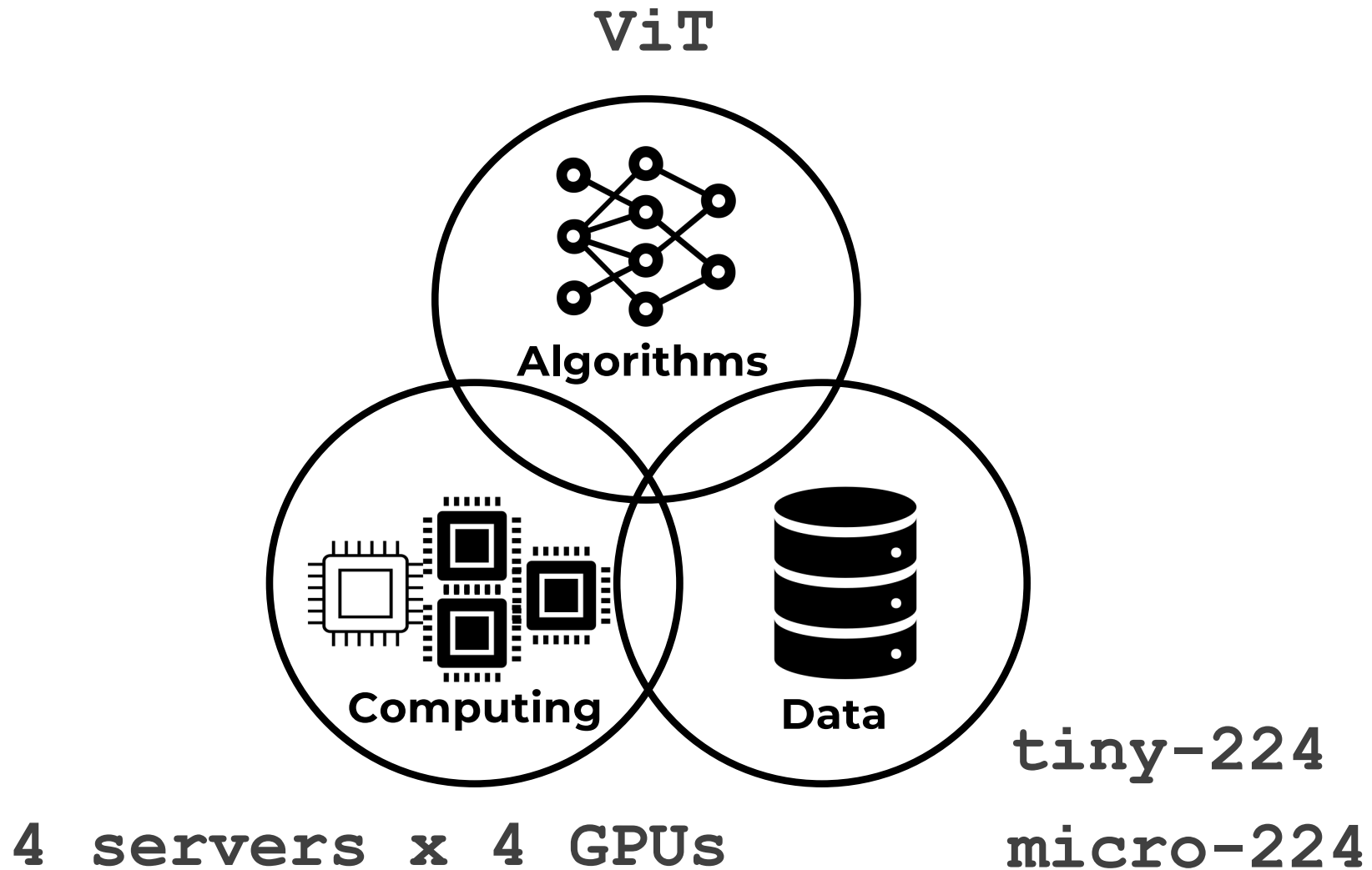
■ Also restrict:

- model checkpoint saving
- metrics logging
- validation printouts



Setting Up for DDP Experiment Execution

Illustrative Case Study



File Setup on MareNostrum 5

- **Datasets location:**

/gpfs/projects/nct_345

```
$ tree -L 2 datasets
datasets
|-- micro-224
|   |-- train
|   `-- val
`-- tiny-224
    |-- test
    |-- train
    `-- val
```


File Setup on MareNostrum 5

■ Home directory: Code from GitHub (*)

```
|-- Chapter.11.slurm-files
|   |-- Ex00.BenchmarkModels.slurm
|   |-- Ex01.MaxBatchSize.slurm
|   |-- Ex02.NumWorkers.slurm
|   |-- Ex03.OptDataLoaders.slurm
|   |-- Ex04.OptDataLoadersViT.slurm
|   |-- Ex05a.DataTypes.fp32.slurm
|   |-- Ex05b.DataTypes.fp16.slurm
|   |-- Ex05c.DataTypes.bf16.slurm
|   |-- Ex06.JIT.slurm
|   |-- Ex07.train.optimizado.slurm
|-- Chapter.12.slurm-files
|   |-- ddp_job_tiny_1n_1g.slurm
|   |-- ddp_job_tiny_1n_2g.slurm
|   |-- ddp_job_tiny_1n_3g.slurm
|   |-- ddp_job_tiny_1n_4g.slurm
|   |-- ddp_job_tiny_2n_4g.slurm
|   |-- ddp_job_tiny_4n_4g.slurm
|   |-- ddp_job_tiny_8n_4g.slurm
|-- benchmark_models.py
|-- custom_model.py
|-- dataset.py
|-- train.py
|-- train_ddp.py
|-- utils.py
```

(*)

local laptop:

```
cd <dir>
git clone https://github.com/jorditorresBCN/HPC4AIbook.git
cd HPC4AIbook
ls
.
.
.
scp -r laptop_dir user@transfer1.bsc.es:MN5_dest_dir
```

Goal of this example

- **Experiments focus on performance**
 - We can run all experiments by editing only the SLURM script.
 - Python code remains fixed.
- **What Changes Compared to Single-GPU?**
 - Distributed initialization.
 - Rank/world-size awareness.
 - GPU assignment per process.
 - DistributedSampler for dataset partitioning.
 - DDP model wrapping.
 - Metric aggregation across processes.
 - Rank-aware logging.

Key differences introduced for DDP

■ Distributed Initialization

```
from torch.distributed import init_process_group  
init_process_group(backend="nccl")
```

- Initializes the process group before any model creation.
- NCCL backend ensures high-performance GPU communication on H100.

Key differences introduced for DDP

- Reads global distributed metadata from environment:

```
world_size = int(os.environ["WORLD_SIZE"])  
rank = int(os.environ["RANK"])
```

- These values determine:
 - Which GPU each process uses
 - How to partition the dataset
 - Synchronization of all replicas

Key differences introduced for DDP

■ Device Assignment Per Process

```
device_id = rank % torch.cuda.device_count()
```

- Each process is “pinned” to one GPU.
- Prevents conflicts between processes on the same node.

■ Distributed Data Loading

```
from torch.utils.data.distributed import DistributedSampler  
  
train_sampler = DistributedSampler(train_ds, num_replicas=world_size,  
                                  rank=rank, drop_last=True)
```

- Ensures each process sees a unique shard of the dataset.
- Shuffles data consistently across processes.
- Prevents duplicated gradients and improves convergence.

Key differences introduced for DDP

■ Wrapping the Model with DDP

```
model.to(device_id)
model = DDP(model, device_ids=[device_id])
```

- Automatically synchronizes gradients during backward().
- Replicates the model across all GPUs.
- Maintains consistency between all replicas after each step.

Key differences introduced for DDP

■ Metric Aggregation Across GPUs

```
from torch.distributed import all_reduce, ReduceOp

accuracy = torch.tensor(accuracy_score(references, predictions), device=device_id)
all_reduce(accuracy, op=ReduceOp.AVG)
```

- Validation runs locally on each GPU.
- Aggregates metrics to obtain global accuracy.
- Ensures reliable, synchronized evaluation.

Key differences introduced for DDP

■ Rank-Aware Logging

- Why?
 - DDP spawns *one process per GPU*.
 - If all print → massive clutter.
- Only rank 0 handles:
 - Logging
 - Checkpoints
 - Summary metrics
- Produces clean output in SLURM logs.

Logging and Output

- Example output

```
INFO - ### model_name: vit
INFO - ### num_workers: 8
INFO - ### batch_size: 64
INFO - ### mixed_precision: bf16
INFO - ### compile: True
INFO - ### optimizer: sgd
INFO - ### dataset: micro-224
INFO - ### Total number of parameters: 631M
INFO - [EPOCH: 3] Accuracy: 0.431 | Validation throughput: 512 imgs/s
INFO - Training throughput: 1216 imgs/s
INFO - GPU memory reserved: 29.8 GB
```

Slurm Scripts

```
#!/bin/bash
#SBATCH --job-name=ddp_tiny_2nodes_4gpus
#SBATCH --chdir=.
#SBATCH --output=./results/R-%x.%j.out
#SBATCH --error=./results/R-%x.%j.err
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --gres=gpu:4
#SBATCH --cpus-per-task=80
#SBATCH --time=01:30:00
#SBATCH --account=<account>
#SBATCH --qos=acc_debug
#SBATCH --exclusive
```

■ Container Setup

```
module purge
module load singularity
SINGULARITY_CONTAINER=/gpfs/apps/MN5/ACC/SINGULARITY/SRC/images/nvidiaPytorch24.07
```

Slurm Script

- Rendezvous Setup for torchrun

```
MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
MASTER_PORT=6000
```

- Launching the DDP Job with torchrun

```
LAUNCHER="torchrun \  
  --nproc_per_node=4 \  
  --nnodes=2 \  
  --node_rank=$SLURM_PROCID \  
  --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT \  
  --rdzv_backend=c10d"  
  
PYTHON_FILE="./train_ddp.py"  
export CMD="$LAUNCHER $PYTHON_FILE $PYTHON_ARGS"  
SINGULARITY_ARGS="--nv $SINGULARITY_CONTAINER"
```

Slurm Script

- **executed inside the container with:**

```
SRUN_ARGS="--cpus-per-task $SLURM_CPUS_PER_TASK --jobid $SLURM_JOB_ID"  
srun $SRUN_ARGS singularity exec $SINGULARITY_ARGS bash -c "$CMD"
```

Slurm Script

■ Training Parameters via Shell Variables

```
MODEL="vit"
DATASET="./datasets/tiny-224"
EPOCHS=5
BS=128
NW=10
OPTIM=sgd

PYTHON_ARGS="--model_name $MODEL \
             --dataset $DATASET \
             --num_epochs $EPOCHS \
             --batch_size $BS \
             --eval_batch_size $BS \
             --num_workers $NW \
             --optimizer $OPTIM \
             --mixed_precision bf16 \
             --compile"
```


How Many Processes Are Running?

- **Example case:**

2 nodes × 4 GPUs per node = 8 GPUs total

- SLURM assigns 1 task per node.
- Each task gets access to 4 GPUs.
- torchrun launches 4 Python processes per node.
- Total = 8 synchronized training processes.



Experimental Methodology

Common setup for all experiments

- **Model: Vision Transformer (ViT)**
- **Data: (Imagenet) Tiny-224 , Micro-224**
- ***Python Script: train_ddp.py***
- **Batch scripts: variants of *ddp_job_tiny_2n_4g.slurm***
- **Only variables changed:**
 - number of GPUs
 - dataset size
- **Why this uniform setup?**
 - Ensures *fair and controlled* comparisons.
 - Isolates the effect of parallelism, not hyperparameters.

Understanding Batch Size in DDP

- **Key concept:**
DDP keeps the *local* batch size fixed on every GPU.

- **Therefore:**

$$\text{global_batch_size} = \text{local_batch_size} \times \text{number_of_GPUs}$$

- **Implications:**

- Increasing GPUs increases global batch size automatically.
- Training becomes more data-parallel but also less noisy (larger batches).
- Throughput improvements should be interpreted relative to *global* batch size.

Throughput Measurement Method

- **To obtain consistent results:**
 - Throughput is computed per epoch (images/s).
 - We exclude epoch 1 from averages because:
 - JIT compilation overhead
 - warm-up of kernels and caches
 - initial dataloader ramp-up
 - Final throughput = average throughput of epochs 2..N.
 - This mirrors standard practice in benchmarking PyTorch workloads.

Scalability Goals

- **We study: Strong Scaling**
 - Fix workload (dataset + model)
 - Increase number of GPUs
 - Measure speedup and efficiency



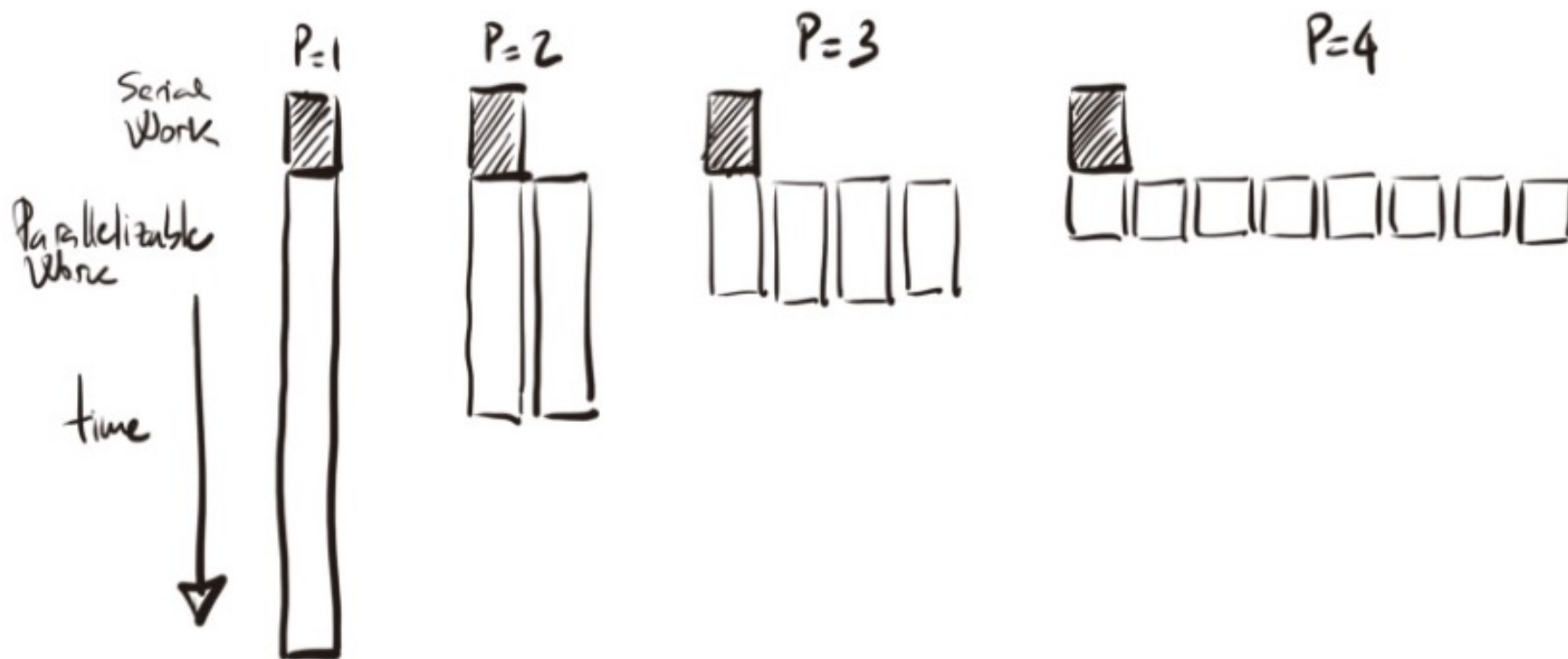
Amdahl's Law (recap)

Speedup Bounds and Scalability Models

- **Two classical models for speedup limits in HPC:**
 - **Amdahl's Law** (1967)
 - **Gustafson-Barsis's Law** (1988)
- **Amdahl: fixed-size problem → speedup is bounded.**
- **Gustafson: growing problem size → speedup can scale nearly linearly.**

Amdahl's Law

- Intuition visual example:



Amdahl's Law

- Let f = sequential fraction (cannot be parallelized).
- Maximum speedup with p processors:

$$Speedup(p) = \frac{1}{f + \frac{(1-f)}{p}}$$

- Even with infinite processors, maximum speedup is:

$$S_{max} = \frac{1}{f}$$

Amdahl's Law

- **Example**

- 90% parallel, 10% sequential, $p = 8$:

$$Speedup(8) = \frac{1}{0.1 + 0.9/8} \approx 4.7$$

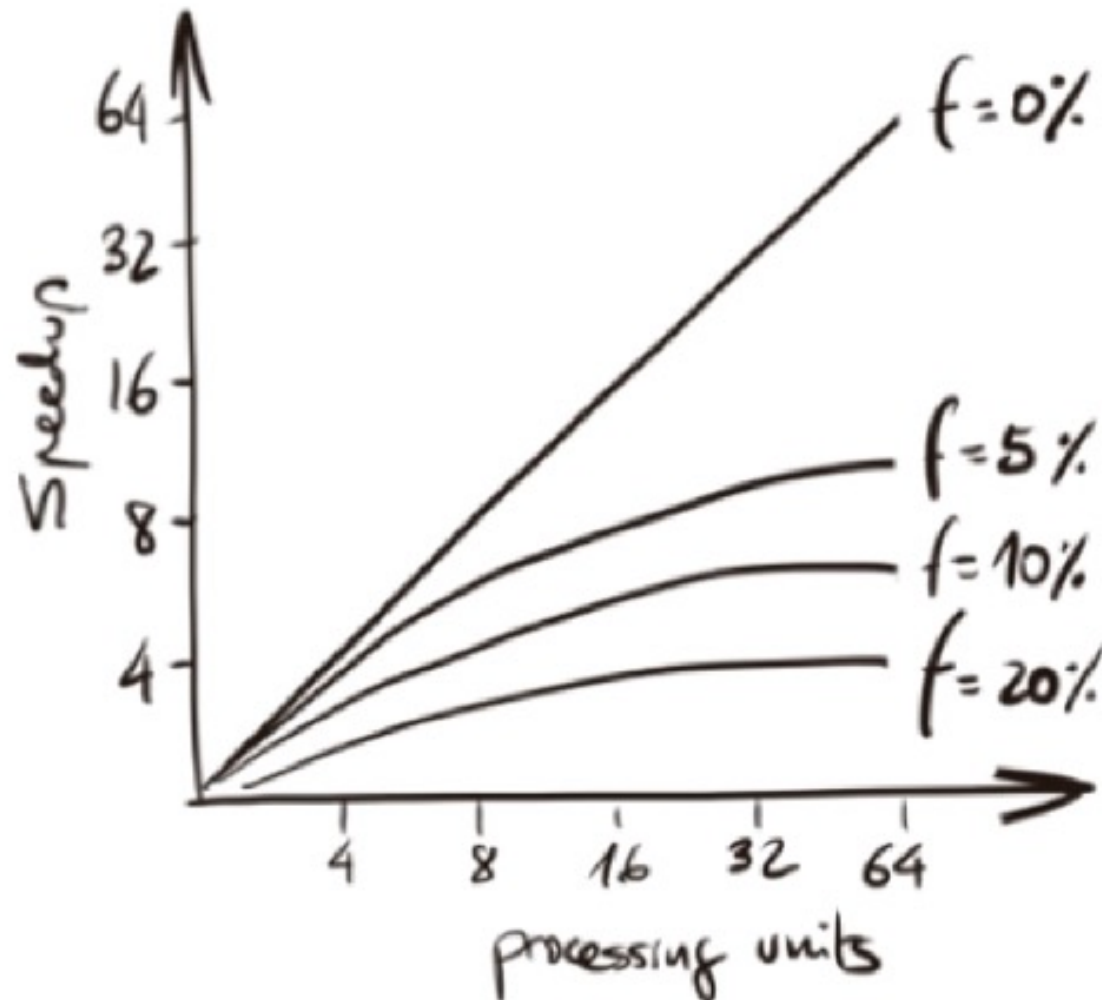
- **Limit with infinite processors:**

$$\lim_{p \rightarrow \infty} \frac{1}{f + \frac{1-f}{p}} = \frac{1}{f}$$

$$Speedup(\infty) = \frac{1}{0.1} = 10$$

Amdahl's Law

- Takeaway: sequential fraction sets the ceiling.



Amdahl's Law

■ Example 2: Matrix-Vector Multiplication:

- The serial part takes 49.13 microseconds, while the parallel portion takes 2.90 microseconds.

→ Total time: $49.13 + 2.90 = 52.03 \mu s$

- Sequential fraction:

$$f = \frac{49.13}{52.03} \approx 0.944$$

- Maximum speedup with infinite processors:

$$S_{max} = \frac{1}{0.944} \approx 1.059$$

- ## ■ Important takeaway: parallelization has almost no benefit when most of the code is sequential.

Limitations of Amdahl's Law

- **Assumes:**
 - Fixed-size problem, identical processors, linear speedup.
- **Does not capture modern systems:**
 - Heterogeneous architectures (CPU + GPU).
 - Cache/memory effects, compiler optimizations.
 - Different algorithms may achieve better-than-expected results.
- **Still: fundamental model and a good starting point for students.**



PRACTICAL WORK

Task 12.1 – Task 12.4

- **This time, the workflow is intentionally more autonomous:**

- You are expected to carefully read **Section 12.4** on throughput and scalability.
- Follow the reasoning, explanations, and figures to understand how performance evolves when scaling across GPUs.
- Use these insights to guide the execution and interpretation of the 4 required Tasks.

- **Your mission:**

- Run the experiments, analyze the results, and justify your conclusions using the concepts introduced throughout the Section 12.4.

Pi: Parallel Model Training with DDP

- **Tasks included:**

- task 12.1, 12.2, 12.3, 12.4

- **Deliverable & Evaluation:**

- Upload a single PDF (per group) to the racó@FIB intranet, containing as many slides as you need per task to clearly express what is requested in each one.

- **In class (evaluation day):**

- One group (chosen at random) will give a clear, concise, and straight-to-the-point presentation.
 - However, do not worry about timing — it does not need to follow a strict “elevator pitch” style as in previous labs.
 - The goal here is pedagogical, allowing time to discuss results and reflect on what has been learned.

SA-MIRI Course Schedule (19/11 Tentative)

day	Wednesday	day	Friday
19/11	12. Parallelizing Model Training with Distributed Data Parallel (Pi) Ph/Pi Presentation Pg	21/11	Pi
26/11	Honoris Causa Dr. Oriol Vinyals (no class)	28/11	Pi
03/12	13. Introduction to Large Language Models 14. End-to-End Large Language Models Workflow Presentation Ph	05/12	Pk
10/12	15. Exploring Optimization and Scaling of LLMs Presentation Pi	12/12	Pk*
17/12	Presentation P1 (16. Looking Forward: HPC and AI Futures) All groups will present	19/12	Pk* “attendance not required (for students returning home)”

Lab Pg presentation

- 1 group **will be randomly chosen**
 - We'll sum 4 numbers from randomly chosen students and use the '%' function with the total number of students to find the winner in the list.

```
>>> nums_to_add = ...+...+...+...  
>>> winner= nums_to_add %  
num_students +1  
>>> print (winner)
```

1	ALONSO GARCIA, MARTÍ;
2	ANDREU GERIQUE, DAVID;
3	ARNABOLDI, LEONARDO;
4	ARÓSTEGUI GARCÍA, ALBERTO;
5	AUBERTIER, THOMAS;
6	CRAMER, TIM;
7	DAHLBOM, IRMA;
8	DÍAZ CALDERÓN, MARC;
9	ESCOLÀ CURCÓ, POL;
10	HENLE, PATRICK;
11	HÖPFL, LEON;
12	KARG, CHISTIAN;
13	KOCH ENES, ANTONIO MANUEL;
14	KOLEVA, ELIZABET;
15	LÓPEZ CASTILLÓN, PAU;
16	MENADOZ, VÍCTOR;
17	MOLDOVEANU, ANDREI;
18	OLIVERA LOYOLA, JUAN JOSE;
19	PAXTON, LAURA;
20	RABADAN ARROYO, RAÛL;
21	RIOS LÓPEZ, ADRIÁN;
22	RITTER, MAXIMILIAN FALK;
23	SELIGA, JAKUB;
24	SIMON, WILLIAM;
25	SOLDERA GIRELLI, VALÉRIA;
26	WALSH, ERIC;